Assignment 2 - Report

Devid Duma
dduma19@epoka.edu.al
Epoka University - Theory of Computation (CEN 350)

Abstract

This is an improvement over the first Lempel-Ziv-78 implementation. The first implementation achieved a compression rate of $144\% - 150\%$, which is good but not great. In this new submission I am going to show a better implementation for Lempel-Ziv which can achieve compression ratios of over 238% (sometimes over 270%) for DNA sequencing.

Keywords – Lempel-Ziv-78 algorithm [1].

1 Introduction

C++17 For this implementation I switched to C++17 to make use of it's superior IO streams and buffers.

Repository I did not upload the repository to repl.it, since this website has poor support for C++17, which is used in the source code. That being said, I chose to rather upload the source code via a zip file at Google Classroom.

Procedure This submission is meant to be only an improvement on the first submission. Since finals are approaching, I am not going to go over all the steps, rather just showcase the new ideas in this new implementation.

Dataset I am going to use the Pizza&Chili Corpus database as a reference for our DNA sequence. In this assignment I used the sequence called *"dna.50MB"*, the 50MB version of the larger 300MB sequence. This proved to be more than enough for our needs. The download link is provided here http://pizzachili.dcc.uchile.cl/texts/dna/

2 Algorithm

2.1 Compression algorithm

Implementation I implemented the algorithm as was described in the class CEN350, with some tweaks, this time in *C++17*. First let's have a look at the code:

C++17 code

```
class LZ78 {
private:

public:
    map<string, long> table;
    vector<tuple<long, char>> compressed_data;

    LZ78() {
    }

    void compress(const string& data) {

    long pos = 0;
    while(pos < data.length()) {

        // find if string exists in dictionary
        long length = 0;
        long id = 0;

        while(pos + length + 1 < data.length() && this->table.find(data.substr(pos, length + 1)) != this->table.end()) {
            length++;
            id = this->table[data.substr(pos, length)] + 1;
        }

        // save next substring to table
        this->table[data.substr(pos, length + 1)] = this->table.size();

        // move pos to += length + 1
        pos += length + 1;
        char next_char;
        // next_char of tuple <id, next_char> is at new position
        if(pos > data.length()) {
            next_char = NULL;
        } else {
            next_char = data[pos - 1];
        }

        this->compressed_data.emplace_back(tuple(id, next_char));
    }
}

static string writeSegment(long x) {
    // I chose modulo 127 rather than 128, to avoid char(-1) == EOF
    int base = 128;
    string x_base;

    if(x > 0) {
        while(x > 0) {
            x_base += char(x % base);
            x = x / base;
        }
    } else {
        x_base = char(0);
    }
    // Flip range [0 ; 127] to [-1 ; -128]
    x_base[x_base.length() - 1] = (x_base[x_base.length() - 1] + 1) * (-1);

    return x_base;
}

static string writePair(long id, char next_char) {
    string id_base = writeSegment(id);

    string result;
    if(next_char != NULL) {
        result = id_base + next_char;
    } else {
        result = id_base;
    }

    return result;
}

string printCompressed() {
```

```
        string compressed_data_string;

        for(tuple pair : this->compressed_data) {
            compressed_data_string += LZ78::writePair(get<0>(pair), get<1>(pair));
        }

        return compressed_data_string;
    }
};
```

Output format The output format of this new implementation is similar with the last implementation: *id* and *next_character* are printed sequentially without any delimiting character between them. There is also no delimiting character between tuples.

This time though, if the *id* is 0, then the *id* **will** be printed.

Encoding numbers as characters This is the most important change to this new implementation. One thing I noticed over the old implementation was that when printing out numbers with base 10 in the output file, we are wasting a lot of bits.

In this new implementation, numbers are outputted with base 128, so 2^7. Characters in files are encoded with 8 bits, and 7 of those bits are now used to save the number. If 7 bits are not enough, the next octets of bits (characters) will contain the remaining information of the number.

The remaining bit in each octet (the first bit of the character) is used to denote if that character is the last character representing that number. If it is the last character, it's first bit will have value 1 and 0 otherwise.

2.2 Complexity

The Lempel-Ziv complexity was first presented in the article *On the Complexity of Finite Sequences (IEEE Trans. On IT-22,1 1976)*[2]. This complexity measure is related to Kolmogorov complexity, but the only function it uses is the recursive copy (i.e., the shallow copy).

The underlying mechanism in this complexity measure is the starting point for some algorithms for lossless data compression, like LZ77, LZ78 and LZW. Even though it is based on an elementary principle of words copying, this complexity measure is not too restrictive in the sense that it satisfies the main qualities expected by such a measure: sequences with a certain regularity do not have a too large complexity, and the complexity grows as the sequence grows in length and irregularity.

The Lempel-Ziv complexity can be used to measure the repetitiveness of binary sequences and text, like song lyrics or prose. Fractal dimension estimates of real-world data have also been shown to correlate with Lempel-Ziv complexity [3][4].

2.3 Decompression algorithm

Implementation The decompression algorithm is shown below.

C++17 code

```

class LZ78Dec {
private:
    string data;

public:
    vector<string> table;
    int counter = 0;

    explicit LZ78Dec(string data) {
        this->data = std::move(data);
    }

    string decompress() {
        string decompressed_data;

        while(!this->data.empty()) {
            tuple pair = readPairFromData();
            long id = get<0>(pair);
            char next_char = get<1>(pair);

            // if id = 0, then no need for table lookup
            if(id > 0) {
                decompressed_data += this->table[id - 1];
                // add to table
                this->table.emplace_back(this->table[id - 1] + next_char);
            } else {
                // add to table
                this->table.emplace_back(string(1, next_char));
            }

            if(next_char != NULL) {
                decompressed_data += next_char;
            }
        }

        return decompressed_data;
    }

    tuple<long, char> readPairFromData() {
        long base = 128;

        long pos = 0;

        long id = 0;
        long power_up = 1;
        while(pos < this->data.length()) {
            if(this->data[pos] < 0) {
                id += (this->data[pos] + 1) * (-1) * power_up;
                break;
            } else {
                id += (this->data[pos]) * power_up;
            }

            power_up *= base;
            pos += 1;
        }
        pos += 1;

        char next_char;
        // next_char and remove tuple
        if(pos < this->data.length()) {
            next_char = this->data[pos];
            this->data = string(this->data.begin() + pos + 1, this->data.end());
        } else {
            next_char = NULL;
            this->data = "";
        }

        this->counter++;
        return tuple(id, next_char);
    }
};
```

Description The decompression algorithm follows the same logic used in the compression algorithm regarding numbers. Since the first bit of a character denotes the end of the number, decompression is possible.

Correctness The compression and then decompression of some input DNA sequence always results in the original sequence. The correctness of the compression and decompression algorithm is therefore assured.

3 Experiment and Results

Task 1 For this task we have to count the number of tuples that were outputted as a result of the compression. I tweaked the code to make it automatic. Nevertheless, the number of tuples outputted, of course, does not change from the last implementation.

For the compressed file *dna.50MB.lines1.lz78* from one line of the original file, there are 18531 tuples in it. For the compressed file *dna.50MB.lines5.lz78* from five lines of the original file, there are 101088 tuples.

Task 2 Here we have to make sure the files are the same. For this purpose I created a small script to check if the original file with the decompressed file has the same content. The script is called ***task2.cpp***. It returned True for both input files, so the compression algorithm is lossless.

Task 3 The data compression ratio is calculated as follows:

$$compression\ ratio = \frac{uncompressed\ size}{compressed\ size}.$$

For the file *dna.50MB.lines1* we have:

$$compression\ ratio = \frac{129KB}{54KB} = 238,88\%$$

For the file *dna.50MB.lines5* we have:

$$compression\ ratio = \frac{831KB}{349KB} = 238,11\%$$

I also tried with a 17MB version file of the DNA sequence. You can see the picture below:

dna.50MB.lines100	17.424 KB
dna.50MB.lines100.lz78	6.397 KB

So, for the file *dna.50MB.lines1* we have:

$$\mathbf{compression\ ratio} = \frac{\mathbf{17424KB}}{\mathbf{6397KB}} = \mathbf{272,37}\%$$

As we can see this is a very good result.

I tried to run the Huffman python script on the compressed files but I could not make it work. The Huffman python script I used before shows this error:

UnicodeDecodeError: 'charmap' codec can't decode byte 0x8f in position 265: character maps to ¡undefined¿

I am not sure how to fix that. It might be because I specified to the *lz78.cpp* to treat the output file as a binary file.

Nevertheless, I expect that the Huffman encoding here would show a much higher efficiency than before, which was 68.16%.

Since I am using the bits of the compressed file better, I will also have more diversity in the characters (a wider alphabet) written on the compressed file than before. The probability distribution will also be more uniform than before. So the Huffman encoding should be more efficient in this case.

4 Conclusion

This new implementation was a big improvement over the last one. The compression ratio is now at 238% for the small files (compare this with $144\% - 150\%$ before).

The compression ratio grows bigger when the filesize is increased too: for the 17MB file we took as an example here, the compression ratio was 272%.

The compressed files now have a wider alphabet than before. The probability distribution of characters should be more uniform.

The Huffman encoding efficiency should also increase, although I was not able to showcase it because of compatibility issues when reading files.

References

[1] J. Ziv, A. Lempel, *Compression of individual sequences via variable-rate coding*. 10.1109/TIT.1978.1055934.

[2] J. Ziv, A. Lempel, *On the Complexity of Finite Sequences*. 10.1109/TIT.1976.1055501.

[3] Thomas Burns, Ramesh Rajan, *Combining complexity measures of EEG data: multiplying measures reveal previously hidden information*. 10.12688/f1000research.6590.1.

[4] Thomas Burns, Ramesh Rajan, *A Mathematical Approach to Correlating Objective Spectro-Temporal Features of Non-linguistic Sounds With Their Subjective Perceptions in Humans*. 10.3389/fnins.2019.00794.